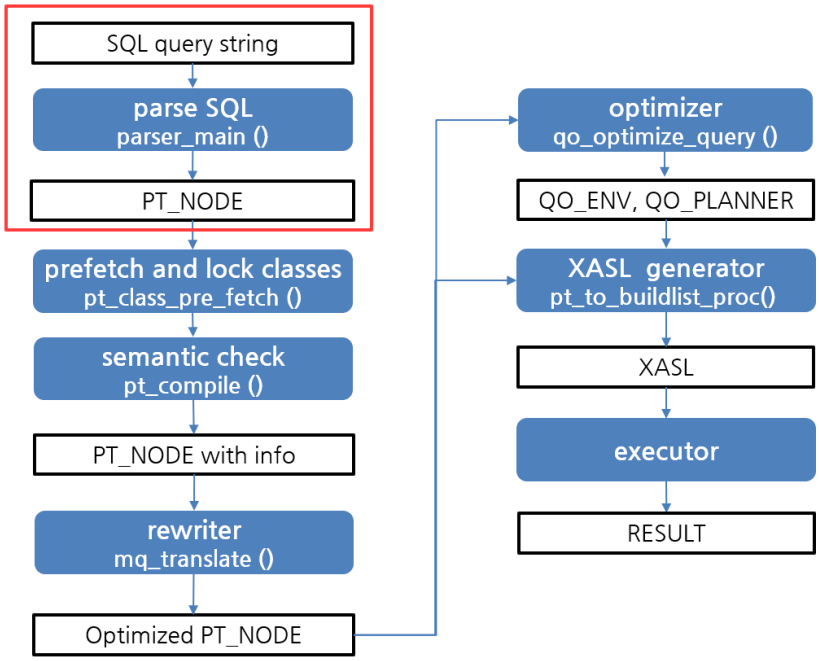
[code_analysis] Parser - Parsing Tree Structure

Created By	Hyung-Gyu Ryoo
Email	hgryoo@cubrid.com
Tags	코드분석
Team	개발2팀
Version	1.0

Overview
PT_NODE 구조
PT_NODE 탐색

Overview

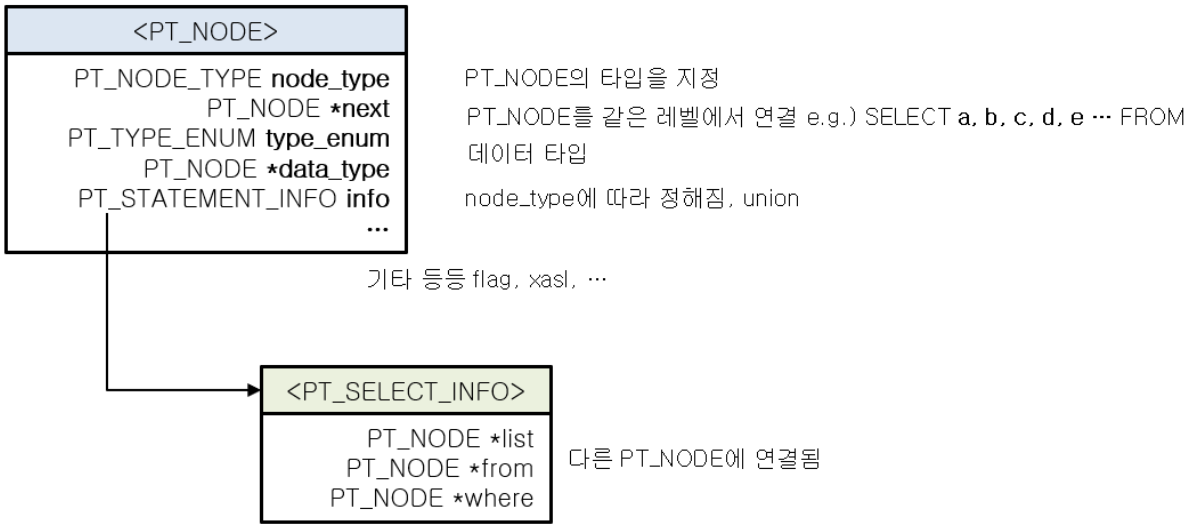


이 부분은 Query Processing 단계 중 가장 처음에 해당한다. Parser에서 입력된 SQL 문자열을 파싱하여 내부 구조인 PT_NODE를 포함한 트리 구조를 만들어내는 것을 목표로 한다.

PT_NODE 구조

PT_NODE는 큐브리드에서 SQL 파싱 트리 구조를 나타내기 위한 구조체다. 큐브리드는 파싱 트리를 다음의 세 가지로 구성한다.

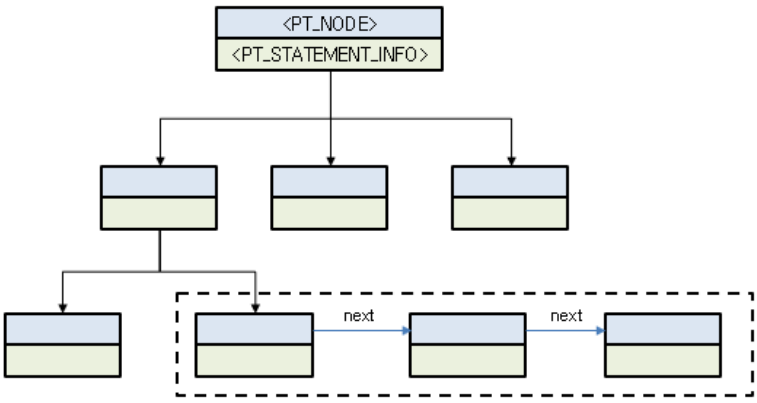
- PT_NODE: 노드 타입과 관련 없는 공통적인 정보
- PT_STATEMENT_INFO: 노드 타입에 따라 가질 수 있는 정보
- Walk functions: 파싱 트리 구조를 순회하며 작업을 수행



SELECT에 대한 PT_NODE 구조 예시

파싱 노드를 구성하는 세 가지 요소가 와닿지 않는다면 위 그림의 예시를 참고해보자. 위 그림은 SELECT에 대한 노드를 보여준다. PT_NODE는 많은 변수를 포함하고 있지만 몇 가지 중요한 변수에 대해 설명해보면 다음과 같다.

- **node_type**: PT_NODE가 어떤 타입을 지정한다. **PT_NODE_TYPE**은 각 노드 타입을 나타내는 Enumeration 타입이다. 예시의 경우 SELECT를 나타내는 것이기 때문에 PT_SELECT를 가진다.
- **next**: PT_NODE가 같은 수준에서 연결되는 경우에 다른 PT_NODE와 연결한다. PT_NODE 구조를 이용해서 일종의 링크드 리스트로 만들어진다. select list, and로 연결된 predicate, from 절에서 테이블 간의 join 등을 표현할 때 사용한다.
- **type_enum, data_type**: 해당 노드의 타입을 나타낸다. type_enum에는 PT_TYPE_INTEGER와 같이 어떤 타입인지 할당하고 data_type에는 그 타입의 구체적인 도메인 정보를 담는다. e.g.) precision, scale, collation 등
- **info**: PT_STATEMENT_INFO는 union 타입으로 각 node_type에 따라 해당하는 구조체가 연결된다. 이 노드에서 구문에 따라 가지는 여러가지 정보 (flag ...) 와 함께 다른 PT_NODE로 연결하여 파싱 트리를 구성한다.



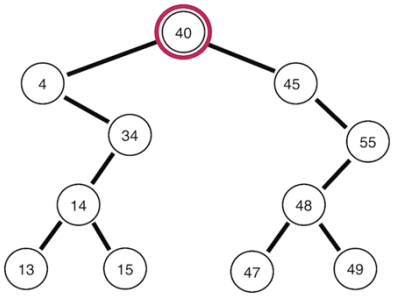
An abstracted view of CUBRID parsing tree

결과적으로 조금 더 멀리서 보면 SQL은 위 그림과 같은 모양의 PT_NODE와 PT_STATEMENT_INFO로 이루어진 트리 형태가 만들어진다.

PT_NODE 탐색

SQL을 큐브리드의 내부 구조, PT_NODE로 이루어진 트리는 앞에서 설명했다. 이 트리 구조를 활용하기 위해서는 순회하기 위한 방법이 필요하다. 다음 그림과 같은 Binary Search Tree의 경우 순회 시 어떻게 노드를 방문할 것인가에 따라 preorder, inorder, postorder로 나뉘기도 한다.

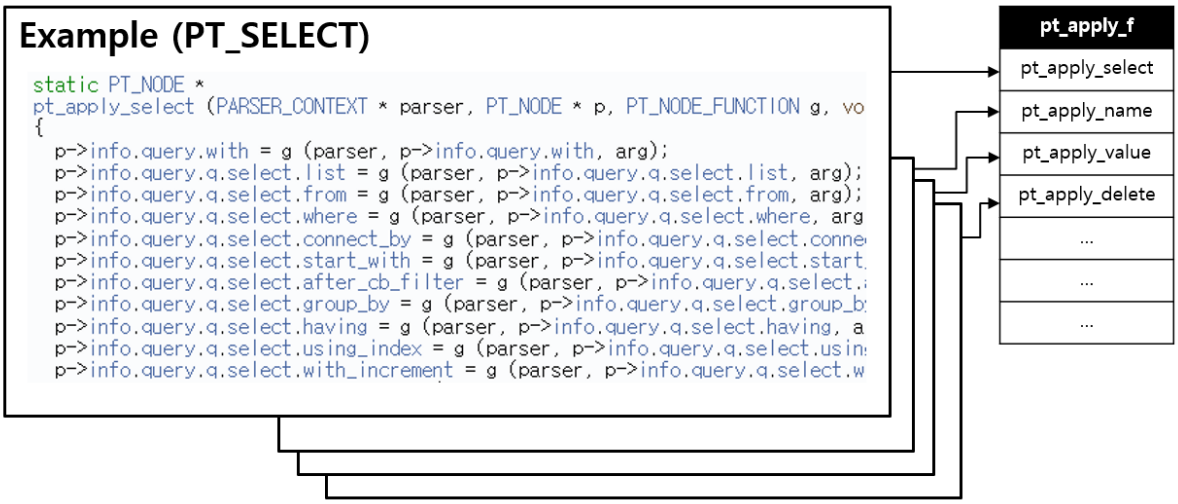
- preorder: 노드를 방문, 왼쪽 서브 트리, 오른쪽 서브 트리
- inorder: 왼쪽 서브 트리, 노드를 방문, 오른쪽 서브 트리
- preorder: 왼쪽 서브 트리, 오른쪽 서브 트리, 노드를 방문



Example figure of Binary Search Tree

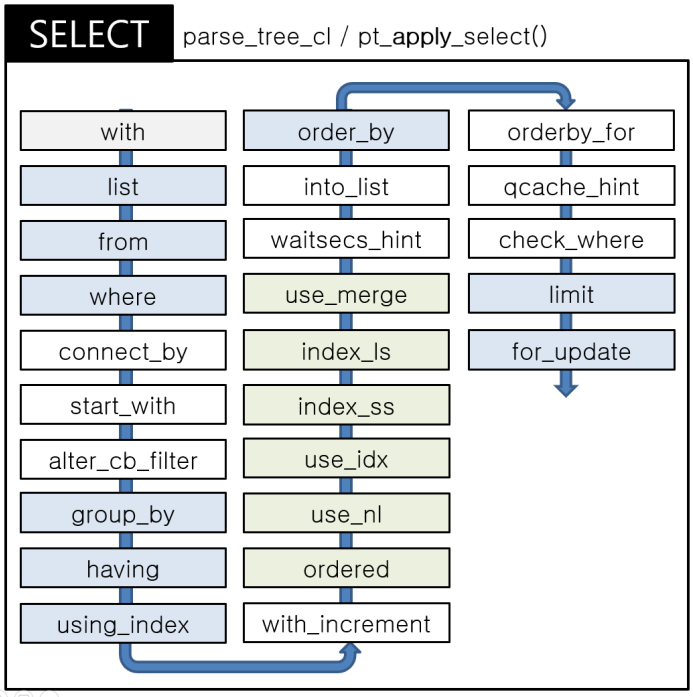
이 장에서는 BST를 설명하려는 것이 아니라, PT_NODE에서도 이와 같이 노드의 종류에 따라 노드를 방문하고 순회하는 순서가 정해져 있다는 것이다. BST와 비교해서 한번 살펴보자

1) Binary Search Tree에서는 노드를 순회할 때 무조건 왼쪽 그리고 오른쪽 서브 트리 순서로 순회하는 것을 볼 수 있다. 이처럼 PT_NODE도 노드의 종류에 따라 순회 하는 순서가 정해져 있다. 다음 그림을 참고하면, pt_apply_f 배열이 각 노드의 종류에 대해서 정해진 순서로 서브 트리를 순회하는 함수를 저장하고 있는 것을 볼 수 있다. pt_apply_f 배열에 저장된 순서는 PT_NODE_TYPE Enum과 같다.



pt_apply_f array which contains function pointers for traversing PT_NODE

예시로 SELECT 노드에 대해서 순회하는 순서를 나타낸다면 다음 그림과 같을 것이다.



2) Binary Search Tree에서는 노드를 언제 방문할 것인가에 따라 preorder, inorder, postorder로 나뉘는 것을 보았다. PT_NODE를 순회할 때도 마찬가지로 방문 순서에 따라 다른 함수를 호출한다. 서브 트리를 순회하는 방법은 정해져 있기 때문에 inorder와 같은 상황은 없을 것이다. 대신 노드를 방문한 다음 순회할지 (like preorder), 순회한 다음 노드를 방문할지 (like postorder) 결정하는 함수가 있다.

```
/* preorder와 같이 노드를 방문한 다음 순회한다 */
PT_NODE *
parser_walk_tree (PARSER_CONTEXT * parser, PT_NODE * node, PT_NODE_WALK_FUNCTION pre_function, void *pre_argument,
      PT_NODE_WALK_FUNCTION post_function, void *post_argument)

/* postorder와 같이 노드를 순회한 다음 방문한다 */
PT_NODE *
parser_walk_leaves (PARSER_CONTEXT * parser, PT_NODE * node, PT_NODE_WALK_FUNCTION pre_function, void *pre_argument,
        PT_NODE_WALK_FUNCTION post_function, void *post_argument)
```

위의 두 함수를 사용하여 파싱 트리를 순회하며 어떤 기능을 수행한다. 함수의 이름을 보면 알 수 있듯이 postorder와 같이 트리를 탐색하면 트리의 leaf node부터 방문하게된다.

그러면 각 노드를 방문하면 어떤 일을 수행하는가? 각 노드를 방문할 때 수행할 동작을 호출할 함수를 injection하여 (함수 포인터를 이용하여) 구현한다. 이 때 노드를 포함한 해당 서브트리에 들어갈 때, 그리고 나올 때 각각 다른 함수를 호출 할 수 있는데 그것이 pre_function과 post_function 인수이다. 각 함수에서 사용할 인수가 각각 pre_argument와 post_argument이다.

앞에서 설명한 것을 수행하는 함수가 바로 pt_walk_private() 다. 이 함수의 구현을 바로 붙여넣었는데 위에서 설명하고 있는 내용을 이 함수를 참고하면 좀 더 쉽게 이해할 수 있을 것이다.

```
 /*
 * from parser_tree_cl.c
 * pt_walk_private () - implements the higher order tree walk routine parser_walk_tree
 *   return:
 *   parser(in):
 *   node(in/out):
 *   void_arg(in/out):
 */
static PT_NODE *
pt_walk_private (PARSER_CONTEXT * parser, PT_NODE * node, void *void_arg)
{
```

```
  PT_WALK_ARG *walk = (PT_WALK_ARG *) void_arg;
  PT_NODE_TYPE node_type;
  PARSER_APPLY_NODE_FUNC apply;
  int save_continue;

  if (node && walk->pre_function)
    {
      node = (*walk->pre_function) (parser, node, walk->pre_argument, &(walk->continue_walk));
    }

  if (node)
    {
      if (walk->continue_walk != PT_STOP_WALK)
  {
    /* walking leaves may write over this. */
    save_continue = walk->continue_walk;

    /* visit sub-trees */
    if (save_continue == PT_CONTINUE_WALK || save_continue == PT_LEAF_WALK)
      {
        /* this is an optimization to remove a procedure call per node from the recursion path. It is the same as
         * calling pt_apply. */
        node_type = node->node_type;

        if (node_type >= PT_LAST_NODE_NUMBER || !(apply = pt_apply_f[node_type]))
    {
      return NULL;
    }

        (*apply) (parser, node, pt_walk_private, walk);

        if (node->data_type)
    {
      node->data_type = pt_walk_private (parser, node->data_type, walk);
    }
      }

    /* visit rest of list first, follow 'or_next' list */
    if (node->or_next
        && (save_continue == PT_CONTINUE_WALK || save_continue == PT_LEAF_WALK || save_continue == PT_LIST_WALK))
      {
        node->or_next = pt_walk_private (parser, node->or_next, walk);
      }

    /* then, follow 'next' list */
    if (node->next && (save_continue == PT_CONTINUE_WALK || save_continue == PT_LIST_WALK))
      {
        node->next = pt_walk_private (parser, node->next, walk);
      }

    if (walk->continue_walk != PT_STOP_WALK)
      {
        walk->continue_walk = save_continue;
      }
  }

      /* and visit this node again */
      if (walk->post_function)
  {
    node = (*walk->post_function) (parser, node, walk->post_argument, &(walk->continue_walk));
  }
    }

  return node;
}
```